Pegasus: защищенное техническое и нетехническое описание

Pegasus: защищенное техническое и нетехническое описание

Тип документа: контролируемое техническое досье в патентном стиле. Версия: один точка один. Режим раскрытия: архитектурное описание, а не инструкция по реализации. Документ описывает, что делает логистическая система Pegasus, почему система отличается, как ведут себя основные контуры управления и какие математические ландшафты формируют изобретение. Документ намеренно не раскрывает исходный код, карты приватных endpoint-ов, внутренние схемы данных, закрытые пороги, размеры развертывания, credential-потоки и константы моделей.

ОПИСАНИЕ

Pegasus является многоролевой логистической операционной системой для поставщиков, операторов фабрик, операторов складов, водителей, ритейлеров и payload-команд. Основная идея достаточно проста. Логистическому бизнесу не нужны шесть разрозненных приложений, которые спорят об истине одного заказа, одного грузовика, одного манифеста или одного платежа. Pegasus рассматривает операцию как общую сеть состояния. Каждая роль видит ту часть сети, которой она владеет; каждое важное действие проходит через управляемый переход; система сохраняет полезность автоматизации, не позволяя автоматизации стереть ответственность.

Техническое изобретение состоит в связывании ролевых интерфейсов, долговечной передачи событий, геопространственного принятия решений, контроля манифестов, финансовой сверки и управления исключениями в одной согласованной операционной модели. Нетехническое изобретение состоит в дисциплине ответственности. Поставщик задает коммерческую политику, фабрика подготавливает пополнение, склад контролирует диспетчеризацию, водитель выполняет проверенную доставку, ритейлер подтверждает спрос и получение, а payload-оператор защищает целостность загрузки. Система не размывает эти обязанности. Она их соединяет.

Документ написан на основе текущей архитектуры репозитория и сопроводительной документации. Реализация использует backend-сервисы на Go, маршрутизацию chi, транзакционную истину на Spanner, Redis cache и Pub/Sub invalidation, Kafka eventing через transactional outbox, ролевые WebSocket-каналы, web-порталы Next.js, нативные Android и iOS приложения и payload-терминал. Эти конкретные технологии упоминаются для привязки раскрытия к инженерной реальности, но защищаемое изобретение шире любого отдельного поставщика или framework-а. Изобретение заключается в модели управления, которая сохраняет согласованность распределенного логистического исполнения при масштабе, частичном отказе и человеческом override.

ПРЕДПОСЫЛКИ

Большинство логистических систем выглядит интегрированным до первого сбоя. Маршрут меняется, на складе дефицит, грузовик недоступен, платеж требует сверки, или водитель прибывает раньше, чем ожидала система. В этот момент слабая архитектура становится видимой. Инвентарь говорит одно. Диспетчеризация говорит другое. Приложение водителя имеет устаревшую работу. Портал сообщает успех, потому что запрос вернул ответ с кодом двести. Финансы находят расхождение позже.

Обычное решение — добавить больше dashboard-ов. Это помогает людям смотреть на беспорядок, но не устраняет его. Более трудная проблема не в презентации. Более трудная проблема — авторитет состояния. Современной логистической платформе нужно знать, какой actor может менять какой объект, какие роли должны быть уведомлены, какое финансовое последствие следует, какие cached views должны быть invalidated, какая автоматизация должна быть приостановлена и какой audit trail доказывает произошедшее.

Pegasus отвечает на это тем, что рассматривает каждый важный workflow как управляемый переход состояния. Система не полагается на дружелюбное тело клиентского запроса для supplier, factory или warehouse scope. Она выводит scope из аутентифицированного role context. Она не рассматривает delivery события как надежду после записи в базу данных. Она связывает durable mutation и durable propagation вместе. Она не позволяет ручному вмешательству соревноваться с автоматической диспетчеризацией. Она использует lock-семантику, чтобы человек мог изменить план, не борясь с машиной.

Нетехнический фон также важен. Операторы не думают микросервисами. Они думают последствиями. Безопасно ли отправить этот грузовик. Запечатан ли этот манифест. Ожидает ли ритейлер поставку. Чист ли платеж. Имеет ли склад право действовать. Полезная платформа должна переводить эти человеческие вопросы в точные технические ограничения, не заставляя человека нести реализационную сложность.

КРАТКОЕ ОПИСАНИЕ ЧЕРТЕЖЕЙ

Набор чертежей основан на карте архитектуры репозитория, machine-readable architecture graph, technology inventory, существующей библиотеке визуальных asset-ов, Mermaid workflow diagram-ах и graph-е backend route composition. Эти материалы используются как описательные evidence для профессиональных патентных чертежей. Они не являются публичной картой реализации и не раскрывают исходный код, приватные endpoint-ы, внутренние схемы, production topology, secrets, thresholds или model constants.

Чертеж инфраструктуры должен показать полный операционный контур снаружи внутрь. Web-порталы, desktop shells, нативные mobile clients и payload terminal входят через Maglev-style routing and protection boundary. За этой границей chi router направляет запросы к backend domain handlers, WebSocket hubs поддерживают живые ролевые каналы, Spanner сохраняет transactional truth, Redis обеспечивает cache и Pub/Sub invalidation, Kafka получает durable events через outbox path, worker-сервисы выполняют automation and reconciliation, а observability связывает активность в traceable system evidence.

Чертеж ingress и request routing должен показать, что clients входят в stateless service layer, а не зависят от sticky sessions или local server memory. Route-composition graph следует представить как сгруппированные contract surfaces для supplier, retailer, driver, payload, factory, warehouse, order, delivery, payment, telemetry, webhook, proximity, catalog, simulation, treasury и infrastructure families. Назначение чертежа — показать, что платформа организована по role and domain contracts, а не по разбросанным endpoint fragments.

Чертеж role responsibility должен показать supplier как commercial and operational owner, factory role как production-side readiness, warehouse role как local fulfillment control, driver role как physical execution, retailer role как demand and receipt confirmation, и payload role как loading integrity. Чертеж также должен показать, что роли соединены управляемыми handoff-ами, а не свободным data sharing.

Чертеж role-surface должен показать, что каждая роль является product row, а не изолированным приложением. Supplier, retailer, driver, payload, factory и warehouse соединяются с backend contracts, web или desktop surfaces, где применимо, Android clients, где применимо, iOS clients, где применимо, terminal surfaces, где применимо, и realtime channels. Этот чертеж должен ясно показать, что parity across a role row является системным свойством, а не косметическим предпочтением доставки.

Чертеж ecosystem workflow должен показать полный business loop. Demand and planning начинаются с supplier policy, catalog, pricing, zones, factories, warehouses и retailer order intent. Fulfillment далее проходит через warehouse stock pressure, factory supply response, payload loading, manifest sealing, driver execution, retailer receipt confirmation, exception handling, analytics, replenishment и следующий demand cycle. Существующие workflow diagram-ы и generated SVG visuals служат основой для этого чертежа.

Чертеж governed transition должен показать business action, входящее в систему как intent, а не permission. Система resolve-ит authenticated identity, выводит supplier или node scope, проверяет policy, оценивает requested state transition, отклоняет stale или replayed attempts, commits state change, emits durable downstream evidence, invalidates affected read models и уведомляет proper role channels. Это control-path view изобретения.

Чертеж transactional outbox and event propagation должен показать database commit и event emission как одну lineage. Mutating handler записывает domain state и outbox record в одной transaction. Outbox relay затем publishes to Kafka, a downstream automation, notification dispatch, realtime refresh, analytics, reconciliation и exception surfaces consume из durable stream. Чертеж должен подчеркнуть, что delivery события не является hopeful side effect после database write.

Чертеж realtime hub должен показать warehouse, retailer, driver, payload, factory, fleet и telemetry channels как role-scoped live surfaces. Redis Pub/Sub fan-out поддерживает multi-pod delivery, а clients должны reconnect или показывать offline state, а не молча замерзать. Чертеж должен показать live channels как operational control surfaces, а не декоративные notifications.

Чертеж auto-dispatch and planning должен показать order demand, warehouse load, driver readiness, vehicle availability, H3-style spatial planning, dispatch preview, manual lock или override behavior, manifest generation, route execution и replenishment feedback. Существующий auto-dispatch pipeline visual дает публичную основу, а codebase route graph поддерживает planning, proximity, supplier logistics и warehouse dispatch relationships.

Чертеж reliability control-plane должен показать priority guard, rate limiting, circuit-breaker posture, Redis invalidation, idempotency, retry behavior, structured logging, metrics, role-restricted failures и stale/offline states как один safety layer. Чертеж должен выражать graceful degradation, а не только uptime. Failed live channel, stale read, out-of-scope action, duplicate mutation attempt или overloaded low-priority request должны стать visible and bounded.

Чертеж financial integrity должен соединить operational completion, payment state, ledger lineage, treasury review, reconciliation и exception handling. Logistics и finance остаются отдельными domains, но их state transitions не должны расходиться. Этот чертеж должен показать, что operational facts и money movement имеют auditable lineage.

Чертеж visual and image register должен сгруппировать public architecture overview, Maglev load-balancing visual, auto-dispatch pipeline visual, reliability control-plane visual, technology-stack composites, omni-code surface artwork, role-feature diagrams, role-relations diagrams, role-surface diagrams, ecosystem workflow diagrams и Pegasus identity logo. Эти визуальные references поддерживают formal architecture, infrastructure, system-flow, technical-stack, role-map и brand-ending pages в PDF, не превращая документ в implementation manual.

Чертеж mathematical landscape должен показать formula families как abstract control surfaces. Engineering and Computer Science охватывает orchestration quality and consistency. Radar, Positioning and Navigation охватывает location-aware confidence. Remote Sensing охватывает noisy observation and scene confidence. Physics and Mathematics охватывает stable optimization. General Physics and Mathematics охватывает uncertainty и boundary между automation and human confirmation.

Чертеж future-vision должен показать assistive autonomy, predictive replenishment, exception anticipation, adaptive routing, supply-lane planning, risk-aware forecasting и operator-governed recommendations. Чертеж должен ясно показать, что recommendations являются bounded proposals с confidence and authority checks, а не unreviewable commands.

Чертеж controlled disclosure должен показать границу между раскрываемым и намеренно скрываемым. Раскрываемый материал включает architecture-level relationships, role behavior, visual system categories, abstract formulas и governed control loops. Скрываемый материал включает executable code, private endpoint mutation maps, schema-level contracts, deployment quantities, secrets, model constants, tuning thresholds и production wiring. Эта граница является частью профессиональной позиции документа.

Подробный реестр фигур

Фигура 1. Контур системной инфраструктуры. Описание: показать все external role surfaces, входящие в protected routing layer, затем показать backend handlers, live hubs, transactional storage, cache invalidation, event relay, worker automation, financial reconciliation и observability как отдельные, но связанные planes. Визуал должен передавать, что Pegasus является distributed operating system, а не single portal.

Фигура 2. Maglev ingress и stateless service routing. Описание: показать requests от web, desktop, mobile и terminal clients, входящие в load distribution boundary, затем поток в stateless backend pods и route families. Визуал должен подчеркивать no sticky sessions, no local authority и fast draining under production rotation.

Фигура 3. Route-composition contract graph. Описание: показать backend route families как grouped domains, включая supplier core, supplier planning, supplier logistics, supplier operations, supplier catalog, supplier insights, retailer, driver, payload, factory, warehouse, order, delivery, payment, telemetry, webhook, treasury, proximity, simulation и infrastructure. Смысл — показать явное ownership role and domain.

Фигура 4. Role-surface matrix. Описание: показать supplier, retailer, driver, payload, factory admin и warehouse admin across backend contracts, web или desktop surfaces, Android surfaces, iOS surfaces, terminal surfaces и realtime channels. Визуал должен сделать cross-client parity видимой.

Фигура 5. Role responsibility and handoff map. Описание: показать supplier policy, питающую factories, warehouses, drivers, retailers и payload operators. Показать factory-to-warehouse replenishment, warehouse-to-driver dispatch, payload-to-driver sealed manifest handoff, driver-to-retailer delivery proof и retailer-to-supplier demand feedback.

Фигура 6. Full ecosystem workflow loop. Описание: показать demand planning, supplier configuration, retailer order intent, warehouse stock pressure, factory supply response, payload loading, driver route execution, retailer confirmation, exception handling, analytics, replenishment и возврат к следующему order cycle.

Фигура 7. Supplier control-plane flow. Описание: показать supplier onboarding, profile, payment configuration, catalog, pricing rules, inventory, warehouse and factory planning, fleet, manifests, dispatch, CRM, returns, analytics и treasury как одну supplier-owned operating chain.

Фигура 8. Retailer commerce capture flow. Описание: показать supplier discovery, catalog browsing, product detail, cart, checkout, payment selection, order creation, active fulfillment, order tracking, receipt confirmation, saved cards, auto-order settings и family member management.

Фигура 9. Driver execution spine. Описание: показать driver login, home-node scope, mission summary, route map, manifest review, QR scan, arrival validation, offload review, payment или cash branch, correction flow, completion и replay-safe retry behavior.

Фигура 10. Payload loading and manifest workspace. Описание: показать truck selection, order list, selected order detail, checklist validation, mark-loaded gate, manifest exception path, seal action, dispatch success state и payload sync event propagation.

Фигура 11. Factory replenishment and transfer loop. Описание: показать stock threshold signal, factory supply request, acceptance, production readiness, transfer manifest, loading bay, driver или payload handoff, warehouse receipt и feedback в supplier network planning.

Фигура 12. Warehouse dispatch and replenishment loop. Описание: показать inventory posture, demand forecast, supply request creation, dispatch preview, dispatch lock acquisition, driver and vehicle availability, supply request updates, live warehouse channel и explicit restricted-state handling.

Фигура 13. Governed state-transition path. Описание: показать request intent entering the system, identity resolution, role scope derivation, object ownership check, policy gate, idempotency replay check, version check, transactional commit, outbox emission, cache invalidation, realtime broadcast и audit lineage.

PEGASUS

Фигура 14. Transactional outbox relay. Описание: показать domain row и outbox row, записываемые inside one transaction, далее relay polling, Kafka publication, event header propagation, downstream worker consumption, notification formatting и published state mark-back.

Фигура 15. Event cascade from order reassignment to notification. Описание: показать route или order reassignment, freeze-lock или dispatch-lock check, outbox event, Kafka relay, notification consumer, formatted message, role-scoped WebSocket broadcast и client refresh.

Фигура 16. H3 geographic sharding and coverage planning. Описание: показать coordinates becoming an H3 cell, neighboring rings supporting coverage, warehouse coverage polygons becoming cell sets, retailer demand mapped to cells и regional data routing или planning decisions using the cell graph.

Фигура 17. Warehouse load penalty curve. Описание: показать utilization на horizontal axis и recommendation penalty на vertical axis, with mild linear region under the load threshold and stronger quadratic region above the threshold. Визуал должен показать, почему почти заполненные warehouses становятся менее привлекательными, даже если географически близки.

Фигура 18. Dispatch clustering and bin-packing. Описание: показать dispatchable orders filtered by eligibility, grouped by H3 cell and adjacent cells, matched to vehicle capacity, assigned by decreasing fit, split when a manifest exceeds practical capacity, and emitted as route or manifest events.

Фигура 19. Optimizer handoff and fallback. Описание: показать planning service preparing a route-solving request, handing it to an optimization worker, receiving a sequence, and falling back to deterministic bin-packing when the optimizer is unavailable. Визуал должен показать continuity under partial dependency failure.

Фигура 20. Freeze-lock consensus between human operator and AI worker. Описание: показать manual intervention acquiring a lock, affected entities being removed from automation queues, operator action completing within a bounded policy window, audit evidence being written, lock release, and automation resuming.

Фигура 21. Home-node principle. Описание: показать drivers and vehicles bound to a home node that may be a warehouse or a factory, with inter-hub transfer manifests acting as the governed boundary when a driver or payload crosses normal node authority.

Фигура 22. Payment settlement and double-entry ledger. Описание: показать order state, gateway authorization, capture or settlement, paired ledger entries, supplier wallet, retailer payment liability, platform fee account, gateway clearing account, and nightly reconciliation detecting anomalies.

Фигура 23. Payment webhook trust boundary. Описание: показать external gateway callback, signature-first verification, idempotency key derivation, typed parsing, transaction update, outbox emission, ledger effect, and rejection path for invalid or replayed inputs.

Фигура 24. Cash collection branch. Описание: показать driver cash confirmation, replay-safe mutation, payment-collected event, ledger posting, supplier credit, platform fee, route completion linkage, and receipt-side confirmation.

Фигура 25. Offline proof and correction path. Описание: показать offline delivery proof capture, local buffering, hash or proof envelope, later synchronization, conflict gate, replay deduplication, line-level correction, refund delta preview, and canonical amendment.

Фигура 26. WebSocket hub cross-pod relay. Описание: показать authenticated socket upgrade, room assignment by role scope, local hub fan-out, Redis Pub/Sub relay, peer-pod fan-out, heartbeat, reconnect state, and fail-open behavior when cross-pod relay is degraded.

Фигура 27. Telemetry and geofence signal loop. Описание: показать driver coordinates, route progress, fleet telemetry channel, admin or supplier map refresh, approach signal, nonblocking proximity event, and guarded completion validation.

Фигура 28. Priority guard and backpressure. Описание: показать incoming traffic classified into payment or auth, dispatch, and read tiers; queue pressure rising; low-priority requests being shed with retry guidance; and high-consequence payment or dispatch paths remaining protected.

Фигура 29. Circuit breaker state machine. Описание: показать external provider calls moving through closed, open, and half-open states, with failure thresholds, fast-fail responses, probe requests, recovery, and observability metrics.

Фигура 30. Idempotency replay gate. Описание: показать mutation request, idempotency key, body hash, first execution and stored response, same-body replay returning the stored result, different-body replay returning conflict, and separate retention windows for API and webhook flows.

Фигура 31. Bento dashboard information mosaic. Описание: показать anchor, statistic, list, control, wide, and full dashboard cells arranged by operational priority. Визуал должен связать data density, loading skeletons, stale states и drill-down behavior, а не decorative dashboard cards.

Фигура 32. Material and native UI consistency map. Описание: показать web and Android using Material 3 discipline, iOS using SwiftUI-native patterns, payload terminal following Material-style operational density, and shared visual tokens flowing through the role surfaces without forcing one platform to mimic another.

Фигура 33. Supplier payment gateway configuration surface. Описание: показать gateway cards for providers, onboarding status, manual setup expansion, update or deactivate controls, supplier-scoped credential governance, and downstream checkout readiness.

Фигура 34. Retail checkout and supplier-scoped order split. Описание: показать one commerce intent entering checkout, payment selection, supplier-scoped order creation, settlement preparation, idempotent placement, and order tracking state.

Фигура 35. Driver map as execution cockpit. Описание: показать live route map, mission markers, selected mission panel, focus controls, scan action, correction action, payment branch, geofence context, and route telemetry in one operational surface.

Фигура 36. Payload dispatch success evidence. Описание: показать sealed manifest result, active truck, secured manifest state, dispatch codes where applicable, reset to new manifest, and machine-readable dispatch release.

Фигура 37. Replenishment threshold and look-ahead loop. Описание: показать safety stock, current stock, breach detection, replenishment lock, pull matrix, predictive push, look-ahead completion, supplier notification, and warehouse/factory response.

Фигура 38. Security and scope enforcement matrix. Описание: показать JWT role, supplier scope, warehouse scope, factory scope, home-node scope, allowed route families, rejected body overrides, structured security log, and operator-visible restricted state.

Фигура 39. Visual asset production map. Описание: показать public architecture visuals, role diagrams, workflow diagrams, technology-stack composites, load-balancer artwork, reliability-control artwork, auto-dispatch artwork, and Pegasus identity assets as inputs that can be paired with the figure register.

Фигура 40. Controlled disclosure boundary. Описание: показать disclosed architecture-level relationships and abstract formulas on one side, and withheld source code, schemas, endpoint mutation maps, credentials, model constants, deployment sizes, and operational thresholds on the other side.

Расширенные текстовые темы для спецификации

Тема A. State authority and role-scoped truth. Спецификация должна объяснять, что каждая client surface является requestor and observer, тогда как backend владеет durable state transitions, role scope и audit lineage.

Тема B. Role-row parity как product invariant. Спецификация должна объяснять, почему роль рассматривается как product row, охватывающая backend, web, desktop, Android, iOS, terminal, realtime и notification consequences.

Тема C. Supplier portal как supplier operating system. Спецификация должна описывать catalog posture, pricing policy, onboarding, billing, gateway configuration, fleet visibility, dispatch control, manifest governance, CRM, returns, analytics, treasury и reconciliation как один supplier-owned control plane.

Тема D. Retailer commerce intent preservation. Спецификация должна описывать, как discovery, cart, checkout, payment, order tracking, saved cards, family members, auto-order settings и receipt confirmation сохраняют одно commercial intent across fulfillment and settlement.

Тема E. Driver execution and proof chain. Спецификация должна описывать route assignment, mission selection, map context, QR scan, arrival validation, offload review, cash or digital payment branch, correction workflow и completion proof как одну physical execution chain.

Тема F. Payload loading как machine-readable readiness. Спецификация должна описывать payload loading не только как ручную складскую работу, но как formal transition, превращающий checklist completion и manifest sealing в dispatch-ready evidence.

Тема G. Factory and warehouse node cooperation. Спецификация должна описывать factory как supply generation и warehouse как local dispatch control, с transfer manifests and replenishment locks, которые соединяют их без стирания node authority.

Тема H. Transactional outbox как atomicity primitive. Спецификация должна описывать domain mutation и durable event creation как одну commit lineage, с publication через relay, чтобы избегать ghost entities и missing downstream notifications.

Тема I. Cache invalidation как correctness, not decoration. Спецификация должна описывать Redis invalidation и Pub/Sub fan-out как coherence mechanism, удерживающий fast read surfaces aligned after mutation.

Тема J. Realtime channels как control surfaces. Спецификация должна описывать live role channels как часть operational truth, включая reconnection, offline state, stale state и fail-open local delivery при relay degradation.

Тема K. Human override with automation standoff. Спецификация должна описывать dispatch locks и freeze-lock behavior как bounded authority transfer от automation к human operator, с auditability and re-engagement.

Тема L. Spatial reasoning with cell-based planning. Спецификация должна описывать geospatial routing, coverage, warehouse selection, retailer demand mapping, dispatch clustering и regional planning как cell-based logic, а не raw coordinate scanning.

Тема M. Financial reconciliation as fulfillment consequence. Спецификация должна описывать ledger entries, payment settlement, cash collection, gateway callbacks, treasury review и anomaly detection как связанные с fulfillment state.

Тема N. Reliability under partial failure. Спецификация должна описывать priority guard, circuit breakers, idempotency, retry behavior, stale reads, offline UI states и fallback route planning как unified reliability posture.

Тема O. Future assistive autonomy. Спецификация должна описывать predictive replenishment, risk-aware routing, adaptive supply lanes, exception anticipation и confidence-bounded recommendations with operator authority preserved.

Расширенные темы формул и логики

Тема формулы 1. Warehouse load penalty. Описание: представить recommendation cost как distance multiplied by a load penalty, где penalty растет мягко при нормальной utilization и более резко по мере приближения warehouse к saturation.

Тема формулы 2. Dispatch fit score. Описание: объединить distance to first stop, vehicle capacity utilization, driver availability, route cohesion, payment readiness и freeze-lock exclusion в один assignment score.

Тема формулы 3. H3 coverage confidence. Описание: оценить service confidence из target cell membership, neighboring cell coverage, warehouse capacity, travel distance и recent dispatch success in the same region.

Тема формулы 4. Route stability loss. Описание: penalize route plans that improve local distance but introduce excessive resequencing, stale instructions, policy violation или driver confusion.

Тема формулы 5. Idempotency replay result. Описание: model replay behavior как function of key, body hash, stored response, endpoint scope и retention window.

Тема формулы 6. Outbox delivery risk. Описание: представить downstream risk как function of committed-but-unpublished age, relay retry count, topic health и consumer lag.

Тема формулы 7. Realtime freshness score. Описание: combine last update age, socket state, reconnect attempts, role room identity и local cache age into visible freshness indicator.

Тема формулы 8. Ledger balance invariant. Описание: представить money correctness как requirement that credits and debits sum to zero per currency and reconciliation window, with anomalies raised instead of hidden.

Тема формулы 9. Priority shedding boundary. Описание: представить request shedding как threshold function of queue depth, endpoint tier, actor rate и retry-after policy.

Тема формулы 10. Human confirmation entropy. Описание: require human review when candidate actions have high ambiguity, high financial consequence, high route disruption или insufficient observational confidence.

ПОДРОБНОЕ ОПИСАНИЕ

Техническое раскрытие

Технически Pegasus является contract-first logistics control plane. Backend владеет durable truth and state transitions. Clients не создают истину locally. Они request, display, confirm и recover. Каждая роль имеет product surface, и каждая role surface выравнивается с backend contract, который ее питает. Это важно, потому что driver app, понимающее route иначе, чем warehouse portal, не является маленькой UX-ошибкой. Это физический operational risk.

Система опирается на additive compatibility, а не на casual renaming. Когда route, event или payload shape меняется, older clients должны продолжать работать, если coordinated migration не говорит иначе. Поэтому архитектура предпочитает stable DTOs, role-row parity, explicit compatibility aliases и clear ownership route-composition packages. Говоря прямо, Pegasus избегает ловушки, где одно приложение тихо уходит вперед, а другие surfaces делают вид, что ничего не изменилось.

Нетехническое раскрытие

Нетехнически Pegasus построен вокруг доверия. Поставщик доверяет, что склад видит ту же бизнес-реальность. Склад доверяет, что водитель получает executable work, а не устаревшую теорию. Ритейлер доверяет, что подтвержденный спрос и receipt правильно влияют на payment and fulfillment. Payload team доверяет, что sealed manifest имеет операционный смысл. Руководство доверяет, что при исключении система может объяснить, кто действовал, с какой authority и что изменилось.

Именно это большинство software descriptions пропускает. Изобретение не только в наличии dispatch, maps, payments, dashboards and apps. Это ожидаемо. Изобретение в том, что эти surfaces ведут себя как одна controlled operating environment, а не как один polished portal рядом с несколькими более слабыми side channels.

Инфраструктура, архитектура, логика, назначение, идея и поток

Инфраструктура разделена на planes, чтобы система могла масштабироваться, не превращая каждый request в борьбу с database. Request traffic проходит через routing and protection layer перед backend handlers. Backend handlers resolve actor identity, scope and policy. Transactional data stores сохраняют durable truth. Cache and Pub/Sub invalidation сохраняют read speed, не принимая stale correctness как normal. Eventing переносит state changes к workers and role surfaces. Live channels держат operational screens fresh. Observability связывает logs, events и operator-visible behavior.

Архитектура role-row oriented. Роль — не один client. Роль — business actor с web, desktop, mobile, terminal, backend, event and notification consequences. Supplier, driver, retailer, payload, factory и warehouse surfaces имеют свои contract rows. Когда backend расширяет role capability, corresponding role clients должны понимать shape, или система должна скрывать capability до появления parity.

Логика следует guarded transition model. Request начинается как intent, not permission. Система resolve-ит actor from authentication, operational scope from claims and node relationships, оценивает whether target object can move to requested state, rejects stale or replayed attempts, commits state change, emits durable event, invalidates affected read models и notifies relevant role channels. Это намеренно строже ordinary CRUD. Logistics is physical. Ошибочный state change может отправить truck, unlock manifest или shift liability.

Назначение — удерживать speed and correctness together. Быстрое software, создающее silent state drift, бесполезно в logistics. Correct software, требующее постоянной manual reconciliation, операционно нежизнеспособно. Pegasus designed to let system optimize by default while giving humans bounded control when default plan no longer matches ground truth.

Идея потока в том, что каждый operational object carries a lineage. Order не просто becomes complete. Он accepted, assigned, loaded, transported, arrived, verified, completed and reconciled through chain of evidence. Manifest не просто sealed. Он assembled, checked, started, sealed, dispatched and exception-handled if reality deviates. Payment не просто settled. Он tied to fulfillment state and reconciled through append-only accounting logic.

Поведение ролей

Supplier role является commercial and operational owner. В текущей product doctrine Admin Portal является Supplier Portal. Supplier owns catalog posture, pricing intent, operational policy, warehouse and factory planning, fleet visibility, analytics, billing setup и exception governance. Supplier role не означает generic platform administrator. Он означает business operator, responsible for its own logistics network.

Factory admin role owns production-side readiness and supply generation. Factory role может prepare transfers, answer replenishment demand, coordinate staff and fleet resources, and keep production-side manifest state coherent with warehouse needs. Его authority node-scoped, потому что factory operator не должен mutate another node только потому, что UI имеет поле.

Warehouse admin role owns local fulfillment control. Warehouse watches stock posture, staff availability, dispatch locks, supply requests, vehicle availability, driver readiness и live operational state. Warehouse role — место, где automation and human override meet most visibly. Warehouse operator может intervene, но intervention must pause conflicting automation and leave an audit trail.

Driver role owns physical execution. Driver receives route work, validates arrival, confirms delivery steps, reports missing items when needed, and completes high-consequence actions through replay-safe requests. Driver role scoped to home node model so execution authority follows driver identity rather than loosely trusted request body.

Retailer role owns demand, receipt and payment-facing confirmation. Retailer places or modifies orders, responds to AI-assisted suggestions when allowed, follows fulfillment state, handles card and cash payment flows, and confirms receipt-side reality. Retailer sits outside supplier ownership in identity terms, but its order and payment events must still reconcile with supplier operations.

Payload role owns loading and manifest integrity. Payload surfaces work around trucks, orders, seals, missing-item reporting, reassignment recommendations and payloader live sync. Эта роль intentionally separate because loading errors become expensive later. System gives payload operators a first-class workflow rather than burying loading inside generic warehouse screen.

Техническая и нетехническая ценность

Техническая ценность Pegasus в том, что он дает distributed logistics shared transition grammar. Role-scoped authorization снижает spoofing risk. Transactional persistence снижает ghost states. Outbox eventing снижает missed downstream updates. Cache invalidation снижает stale read traps. WebSocket channels снижают blind spots in live operations. Idempotency снижает duplicate high-consequence mutations. Version-aware behavior снижает stale replay damage. Эти части не декоративны. Они существуют потому, что logistics breaks at the edges between systems.

Нетехническая ценность в том, что люди могут доверять operating picture. Supplier видит, где растет risk. Warehouse может объяснить, почему dispatch changed. Driver can retry without duplicating an action. Retailer can track fulfillment without needing to understand internal logistics. Payload operator can report loading truth early. Finance can reconcile from the same lineage instead of cleaning up after the system.

Ландшафт формул Engineering and Computer Science

Этот landscape моделирует orchestration quality. Формула записана plain notation, чтобы пережить PDF export и читаться без math renderer.

```
Q_ops = alpha_valid * valid_transition_rate + alpha_sync * role_sync_score - alpha_conflict *
                      conflict_rate - alpha_latency * propagation_delay
```

Формула описывает system goal в engineering terms. Valid transitions should rise. Role synchronization should rise. Conflict rate and propagation delay should fall. Exact weights are intentionally not disclosed because they are implementation-sensitive. Patent-level idea is the use of composite orchestration score that treats correctness, role agreement, conflict and propagation delay as one control surface.

Ландшафт формул Radar, Positioning and Navigation

Этот landscape моделирует location-aware confidence. Logistics decisions often depend on where an actor, vehicle, warehouse, retailer or route actually is, but location readings are imperfect. Поэтому система treats position as confidence problem, not blind coordinate lookup.

```
p_hat(t) = argmin_over_p SUM[k in S(t)] w_k * residual_score(signal_k, p, time_lag_k)
```

Формула states that chosen position estimate is candidate position that minimizes weighted residual error across available signals and time lag. Изобретение не требует one specific positioning vendor. It requires location-sensitive actions be evaluated against confidence model before affecting completion, dispatch, arrival or exception state.

Ландшафт формул Remote Sensing

Этот landscape моделирует operational scene confidence. В logistics observed scene can be incomplete. Signal may be delayed, occluded, low quality or inconsistent with another source. Platform therefore treats scene state as confidence score rather than simple yes or no.

```
C_scene = beta_signal * signal_quality + beta_coherence * cross_signal_agreement - beta_occlusion *
occlusion_penalty - beta_staleness * data_age
```

Формула captures why warehouse, route, driver or manifest state should not be trusted equally under all observation conditions. High signal quality and cross-signal agreement increase confidence. Occlusion and stale data reduce confidence. Protected concept is the use of scene confidence as gate for automation and escalation.

Ландшафт формул Physics and Mathematics

Этот landscape моделирует stable optimization. Routing, replenishment, dispatch and balancing systems can become unstable if they chase every small change. Pegasus uses idea of regularized objective. System seeks good fit to current reality while penalizing instability and policy violation.

```
Loss(x) = lambda_fit * norm2(x - x_hat)^2 + lambda_smooth * norm1(gradient(x)) + lambda_policy *
policy_penalty(x)
```

Это corrected format for formula that previously rendered poorly. It avoids raw LaTeX commands such as backslash mathcal and backslash lambda. In words, system compares candidate plan to observed plan, penalizes unnecessary jagged changes, and adds policy penalty when plan would violate business or safety constraints. Constants are not disclosed because they encode operational tuning.

Ландшафт формул General Physics and Mathematics

Этот landscape моделирует uncertainty. Automation should act when uncertainty is low and ask for human confirmation when uncertainty is high. System can treat decision ambiguity as entropy.

```
Entropy(P) = - SUM[i] p_i * log(p_i), with SUM[i] p_i = 1 and 0 <= p_i <= 1
```

Low entropy means system has clear best action. High entropy means system sees multiple plausible interpretations. Non-technical translation is straightforward. If system is not confident, it should stop pretending and ask the right operator.

Функции будущего видения

Будущая версия Pegasus не должна стать black box, который толкает operators. Более сильный путь — assistive autonomy. System can recommend dispatch batches, replenishment timing, route changes, supply-lane shifts, warehouse territory changes, payment exception handling and staffing adjustments while preserving role authority and auditability.

Next generation features should make risk visible before it becomes a ticket. Warehouse should see likely dispatch bottleneck before drivers wait. Supplier should see forecast drift before inventory collapses. Factory should see replenishment pressure before warehouse starts escalating. Retailer should see fulfillment confidence without decoding internal state. System should get quieter when things are healthy and more precise when they are not.

Patent-relevant idea behind future vision is not just prediction. Prediction alone is cheap. Stronger idea is prediction with governed actuation. Recommendation becomes useful only when system can explain role scope, confidence level, expected consequence, rollback path and audit evidence that will remain afterward.

Дополнительные профессиональные поля

Позиция новизны: Pegasus combines role-row contract integrity, transactional event lineage, geospatial confidence, manifest governance, replay safety and financial reconciliation into one logistics control plane. Novelty strongest where these mechanisms interact rather than where any one mechanism stands alone.

Промышленная применимость: System applies to supplier-led distribution, factory-to-warehouse replenishment, warehouse dispatch, direct-to-retailer fulfillment, payload loading, driver execution, route monitoring, payment reconciliation and exception handling. Same control model can be adapted to regulated delivery, cold-chain logistics, high-value goods, route-sensitive fulfillment and multi-node regional supply networks.

Reliability posture: System designed to degrade in visible ways. Dropped live channel should reconnect or show offline state. Stale view should be labeled rather than trusted. Retry should replay safely rather than duplicate mutation. Manual override should lock affected entity instead of racing automation.

Security posture: Scope must come from authenticated identity and node relationship, not convenient field supplied by client. Mutating actions should be replay-safe, audit-backed and role-bound. External integrations should be verified before body parsing or state mutation. This brief does not disclose secret material, signing details, private endpoints or production topology.

Commercial posture: Platform supports business that needs operational density without losing control. It lets supplier operate at network level, lets node operators act locally, lets drivers execute without guessing, lets retailers trust fulfillment view and lets finance reconcile from same lineage.

Reverse-engineering posture: Direct reproduction from this document alone is intentionally impractical. Description omits executable source, database schema details, private constants, full endpoint maps, model tuning values, credential flows, infrastructure sizing and failure-mode thresholds. Document explains invention well enough for review, but not well enough to clone the working system.

Заключительное профессиональное описание

Pegasus лучше всего понимать как governed logistics state machine, распределенную по нескольким человеческим ролям и software surfaces. Она достаточно техническая, чтобы enforce consistency, достаточно практичная, чтобы соответствовать реальной работе logistics teams, и достаточно осторожная, чтобы не позволять automation quietly take authority it should not have. System wins by keeping truth shared, authority scoped, transitions auditable and future automation answerable to the people who run the operation.

PEGASUS

Intellectual property of The Lab Industries.